

PROPOSAL

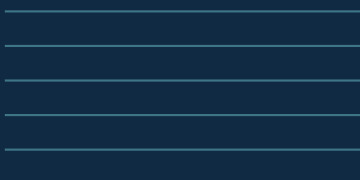
One-Worker Agent-Aware Controller for SGLang

A software-first prototype using three core hooks: agent state, scheduling, and KV-cache control.

Scope	Single SGLang worker; HBM and host-memory tiers
Purpose	Implementation plan for Codex
Date	August 27, 2026

Core result

Prepare important agent requests before their tools finish, then run them at the next safe scheduling point without leaving valuable GPU capacity unnecessarily idle.



1. Executive summary

Build a local, software-only controller around one SGLang worker. The controller receives agent lifecycle information from the harness, influences SGLang scheduling at safe boundaries, and explicitly manages KV movement between GPU HBM and CPU host memory.

The first prototype has three hooks:

- Agent-state hook: tells the controller when a tool starts, its expected completion, when it completes, and when the session ends.
- Scheduler hook: controls request order, background prefill budget, and safe preemption inside the single SGLang worker.
- KV-cache hook: demotes KV from HBM to host memory, prefetches it back to HBM, and releases it when no longer needed.

The central experiment is a high-priority agent that waits on a tool while normal coding and reporting requests share the same GPU. The agent must begin generation within a specified time after the tool completes, while background throughput should remain useful.

Primary hypothesis

Harness knowledge can improve deadline success and HBM efficiency beyond stock reactive scheduling, without requiring new hardware.

Success is not defined as 100 percent GPU utilization. It is defined as maximum useful throughput while admitted critical requests meet their latency targets.

Deliverables

Deliverable	Description
Controller service	Receives lifecycle events, maintains session state, estimates timing, and issues local commands.
SGLang local enforcer	Applies scheduling decisions and KV actions at safe internal boundaries.
Benchmark harness	Runs normal SGLang and controller-enabled SGLang under the same workload.
Results report	Compares deadline success, TTFT, background throughput, HBM occupancy, and KV restoration delay.

2. Scope and design decisions

Version 1 scope

- One SGLang worker on one GPU or one tensor-parallel worker group.
- One controller colocated with that worker or on the same low-latency host network.
- Two KV tiers only: GPU HBM and CPU host memory.
- One critical user-facing agent plus normal and background requests.
- Tool lifecycle events supplied by the harness.
- Software-only changes; no FPGA or custom silicon.

Explicit non-goals for Version 1

- No multi-worker routing or cross-worker KV movement.
- No distributed L3, SSD, or object-storage policy in the first prototype.
- No arbitrary interruption of a running GPU kernel.
- No change to model reasoning, tool selection, token sampling, or output content.
- No absolute deadline guarantee under unlimited offered load.

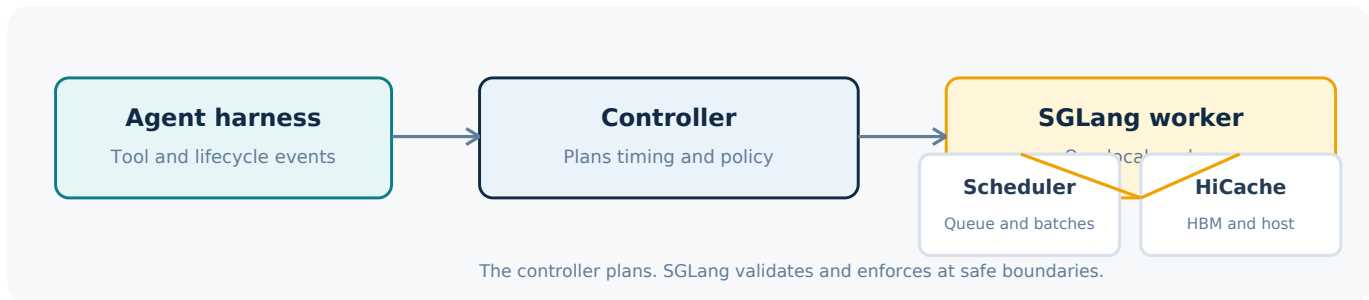
Design rules

Rule	Meaning
Plan outside, enforce inside	The controller decides intent; SGLang validates and applies it at safe boundaries.
Fail open	If controller commands fail or expire, generation falls back to normal SGLang behavior.
Use measured P99 values	Scheduling and prefetch timing use observed tail latency, not assumed averages.
Separate priorities	Execution urgency, HBM residency value, and durable retention value are distinct.
Do useful work	Background work fills spare capacity but yields when a critical deadline is at risk.

Important limitation

The controller can improve predictability, but it cannot create GPU capacity. If accepted urgent work exceeds capacity, admission control, degradation, or rejection is still required.

3. Proposed one-worker architecture



The controller is logically separate from SGLang, but the enforcement path is local. The external controller must never mutate SGLang queue objects asynchronously.

Component responsibilities

Component	Owns
Harness adapter	Emits tool and session events with stable session and prefix identifiers.
Controller state store	Tracks phase, deadlines, tool ETA, prefix identity, and outstanding commands.
Timing estimator	Maintains P50/P95/P99 KV-transfer, scheduling, queue, prefill, and TTFT estimates.
Policy engine	Calculates when to demote, prefetch, restrict background work, and raise urgency.
Local SGLang enforcer	Receives versioned commands and exposes safe scheduler and HiCache hooks.
SGLang scheduler	Maintains the real queue and running batch, validates memory, and executes safe preemption.
HiCache	Performs physical KV copies between HBM and host memory.

Direct queue influence

Yes, the local enforcer can directly influence scheduling. It should do so through a scheduler policy callback, not by allowing the external controller to reach into internal lists.

Recommended scheduler integration pattern

```

decision = controller_policy.plan(
    waiting_queue=waiting_queue,
    running_batch=running_batch,
    kv_state=kv_snapshot,
    now_ms=monotonic_time_ms,
)

sglang_scheduler.validate_and_apply(decision)

```

4. Core hook 1: Agent-state hook

Purpose

Provide the future-facing information SGLang cannot infer from an isolated model request.

Minimum events

Event	Required fields	Controller action
tool_started	session_id, prefix_id, expected completion, uncertainty, response deadline, priority	Enter TOOL_WAIT and evaluate KV demotion.
tool_eta_updated	session_id, updated ETA and uncertainty	Move or cancel the preparation window.
tool_completed	session_id, completion timestamp	Enter READY and immediately raise execution urgency.
session_finished	session_id and prefix_id	Cancel timers and release KV and reservations.

Event contract

Example: tool start event

```
{
  "schema_version": "1.0",
  "event_id": "event-901",
  "session_id": "support-417",
  "prefix_id": "support-417-turn-8",
  "event": "tool_started",
  "expected_completion_ms": 30000,
  "eta_uncertainty_ms": 2000,
  "deadline_after_completion_ms": 100,
  "execution_priority": 10,
  "hbm_residency_priority_while_waiting": 2,
  "host_retention_priority": 10
}
```

Controller state machine

State	Meaning	Default action
ACTIVE	Model work is running or ready.	Normal or elevated scheduling priority.
TOOL_WAIT	No model request is ready; a tool is running.	Demote valuable inactive KV to host if beneficial.
PREPARE	Return window is close.	Prefetch KV and reduce large background chunks.
READY	Tool result is available.	Place resumed request first at the next safe boundary.
FINISHED	Session will not return.	Release KV and cancel pending commands.

Rules

- Events must be idempotent: repeating event-901 must not schedule duplicate copies or reservations.
- Each command must carry an expiry time and session generation so stale commands are ignored.
- Use a monotonic server clock for local timers; convert harness timestamps at ingress.
- An early tool completion must override the predicted ETA immediately.

5. Core hook 2: Deadline-aware scheduler hook

Purpose

Choose the most valuable next GPU work while preserving enough scheduling opportunities for urgent agent resumes.

What the hook may influence

Control	Effect
Effective queue priority	Moves READY critical requests ahead of normal and background work.
Next-batch membership	Ensures the critical request is included in the next feasible batch.
Background prefill budget	Reduces long prefill chunks near an expected critical return.
Safe preemption candidates	Selects lower-priority running requests to retract at a supported boundary.
Background admission gate	Temporarily holds new background work when deadline risk is high.

Scheduling rule

Meet admitted hard deadlines first; protect interactive decode second; fill remaining capacity with preemptible background work third.

A useful urgency calculation is:

Deadline slack calculation

```
remaining_slack_ms = (
    deadline_ms
    - now_ms
    - estimated_remaining_service_p99_ms
)

if remaining_slack_ms <= safety_margin_ms:
    urgency = CRITICAL
elif remaining_slack_ms <= prepare_window_ms:
    urgency = PREPARE
else:
    urgency = NORMAL
```

Example scheduler decision

```
{
  "decision_id": "sched-221",
  "run_first": ["support-417"],
  "defer": ["report-22"],
  "preempt_at_safe_boundary": ["coding-83"],
  "max_background_prefill_tokens": 1024,
  "expires_in_ms": 25
}
```

Safety boundary

The hook cannot stop a GPU kernel mid-execution. It changes work at SGLang scheduling boundaries. Chunked prefill creates more frequent boundaries and therefore improves responsiveness.

6. Core hook 3: KV lifecycle

Purpose

Prevent long tool waits from wasting scarce HBM while preserving the ability to resume without recomputing the full prompt.

Version 1 actions

Action	When used	Required behavior
demote	Tool wait begins and predicted idle time justifies a copy.	Ensure host copy exists, then make GPU blocks reclaimable.
prefetch	Return window approaches or completion-imminent is received.	Allocate HBM pages and restore the requested prefix from host.
release	Session finishes or the branch is cancelled.	Release HBM and host pages and cancel outstanding transfers.

Command examples

```
{
  "action_id": "kv-901",
  "session_id": "support-417",
  "prefix_id": "support-417-turn-8",
  "action": "demote",
  "target": "host",
  "retain_until_ms": 35000
}

{
  "action_id": "kv-902",
  "prefix_id": "support-417-turn-8",
  "action": "prefetch",
  "target": "hbm",
  "execute_at_ms": 29978,
  "expires_at_ms": 30100
}
```

Demotion decision

Demotion is beneficial only when the expected idle interval exceeds the cost of copying out and restoring later. The initial rule can be conservative:

```
demote when:
  expected_tool_wait_p50
  > demote_p99 + restore_p99 + safety_margin
and:
  host_capacity_is_available
and:
  reuse_probability >= configured_threshold
```

Prefetch decision

Use the earliest credible return time, not merely the average ETA. A completion-imminent signal is preferred. If uncertainty is large, either prefetch earlier, keep the KV in HBM during the uncertainty window, or restore on demand when restoration P99 still fits the deadline.

7. End-to-end operating example

Workload

Request	Role	State
support-417	Critical customer-support agent	Waiting 30 seconds for a database tool; 100 ms resume target.
coding-83	Normal-priority coding agent	Processing a 100,000-token repository using chunked prefill.
report-22	Background reporting task	Best-effort work with no hard deadline.

Timeline

Time	Hook	Action
0.000 s	Agent state	Harness sends tool_started for support-417.
0.005 s	KV	Controller requests demotion; HiCache copies KV to host and releases HBM pages.
0-29.9 s	Scheduler	Coding and reporting work use the freed capacity normally.
29.978 s	KV	Controller starts host-to-HBM prefetch using measured P99 restore time and margin.
29.980 s	Scheduler	Background prefill budget drops from 8,192 to 1,024 tokens.
30.000 s	Agent state	Harness sends tool_completed and submits the resumed request.
Next boundary	Scheduler	support-417 is placed first; lower-priority work yields safely.
After admission	Scheduler	Temporary restriction is removed; background work resumes.
Session end	KV	Controller releases the session KV and cancels any stale actions.

Illustrative timing calculation

```
Earliest credible tool return:      30,000 ms
Host-to-HBM restore P99:           12 ms
SGLang setup P99:                   4 ms
Safety margin:                       6 ms
-----
Start preparation at:               29,978 ms
```

These numbers are illustrative. The prototype must measure them on the target GPU and host-memory path before using them for decisions.

8. What SGLang provides and what must be added

Capability	Existing substrate	Controller work
Request ordering	Priority-aware queueing is available when enabled.	Add deadline slack, phase awareness, and a scheduler policy callback.
Safe preemption	Priority preemption and decode retraction mechanisms exist.	Select victims based on deadline risk and restore cost.
Chunked prefill	Configurable chunk sizes already create scheduling boundaries.	Make the effective background budget responsive to a return window.
HBM/host KV movement	HiCache provides the physical copy and storage machinery.	Add explicit, per-prefix demote, prefetch, and release commands.
Prefix matching	Radix cache tracks reusable token prefixes.	Add stable harness-to-prefix/session identity and lifecycle ownership.
Metrics	SGLang exposes queue, request, cache, and latency metrics.	Add controller decision, command, transfer, and deadline metrics.

Likely integration locations

Area	Candidate source location	Change
Request metadata	SGLang request input and internal Req structures	Carry session, deadline, phase, and controller action identifiers.
Scheduling policy	python/sglang/srt/managers/schedule_policy.py	Expose policy snapshot and apply validated queue ordering.
Scheduler loop	SGLang scheduler manager	Apply batch budget and safe preemption at existing boundaries.
Server configuration	python/sglang/srt/server_args.py	Add feature flags, timeouts, and controller endpoint configuration.
HiCache	python/sglang/srt/mem_cache/hiradix_cache.py	Expose targeted demote, prefetch, and release operations.
Observability	SGLang metrics and tracing modules	Record decisions, rejections, expiry, transfer time, and deadline results.

Repository warning

SGLang internals change rapidly. Codex should first pin the exact SGLang commit and inspect the active request, scheduler, and HiCache paths before editing. Candidate locations above are architectural anchors, not a substitute for repository inspection.

9. Recommended software structure

Controller package

```

agentic_controller/
  api.py           # lifecycle event endpoint
  models.py        # versioned events and commands
  state_store.py   # per-session state machine
  timing_estimator.py # P50/P95/P99 estimates
  policy.py        # demote, prepare, urgency rules
  command_bus.py   # local enforcer transport
  metrics.py       # decisions and outcomes

sglang_controller_ext/
  local_enforcer.py # validates and stores commands
  scheduler_adapter.py # queue and batch hook
  kv_adapter.py     # HiCache commands
  capability_api.py  # reports enabled features

tests/
  unit/
  integration/
  workload/

```

Local command contract

Property	Requirement
Versioned	Every event and command contains schema_version.
Idempotent	Repeated event_id or action_id produces no duplicate effect.
Expiring	Every time-sensitive command has an expiry.
Generation-aware	A stale command cannot act on a newer turn with the same session ID.
Capability-checked	Controller queries which hooks are enabled before issuing actions.
Observable	Accepted, rejected, clipped, deferred, expired, and completed actions are counted.
Fail-open	Unsupported or failed actions fall back to normal cache lookup and scheduling.

Recommended transport

For Version 1, use a local Unix-domain socket or in-process queue between the controller and the local enforcer. Do not place a remote synchronous RPC on every scheduler iteration. The controller should send state updates ahead of time; the scheduler reads a local, lock-bounded policy table.

Minimum persisted state

- session_id, session_generation, and prefix_id
- current phase and last accepted event_id
- deadline and earliest/predicted/latest completion window
- execution, HBM residency, and host retention priorities
- current KV tier and outstanding transfer action
- current scheduler restriction and expiry

10. Implementation plan for Codex

Milestone 0: Pin and inspect

- Record the exact SGLang commit, CUDA version, model, GPU, launch flags, and HiCache configuration.
- Trace one request from HTTP ingress through tokenization, waiting queue, batch formation, cache matching, prefill, and decode.
- Locate the current priority, preemption, chunked-prefill, HiCache write, load-back, and eviction paths.
- Create an integration note before editing source.

Milestone 1: Agent-state controller

- Implement versioned event models and an event endpoint.
- Implement the per-session state machine and idempotency.
- Implement tool ETA timers and stale-event protection.
- Emit structured logs and Prometheus-style metrics.

Milestone 2: Scheduler hook

- Carry session and deadline metadata into the internal SGLang request.
- Add a local policy table read at safe scheduler boundaries.
- Implement queue priority, background prefill budget, and safe preemption decisions.
- Preserve stock behavior when the feature flag is disabled or no policy exists.

Milestone 3: KV hook

- Add targeted demote, prefetch, and release commands keyed by session and prefix identity.
- Reuse HiCache page allocation and copy machinery rather than creating a second cache system.
- Track transfer completion, failure, expiry, and freed HBM tokens.
- Fall back to recomputation when the requested prefix cannot be restored.

Milestone 4: Closed-loop timing

- Measure actual transfer, scheduling, queue, prefill, and TTFT distributions.
- Use P99 values and configurable safety margins for preparation timing.
- Update estimates online and compare predicted versus observed deadline outcomes.

Milestone 5: Benchmark and report

- Run identical traces against stock and controller-enabled SGLang.
- Test light load, 70-80 percent load, near saturation, and overload.
- Publish both deadline improvements and background-throughput costs.

11. Experiment and validation plan

Comparison systems

System	Purpose
A. Stock SGLang	Baseline priority, chunking, and HiCache behavior without lifecycle control.
B. Controller scheduling only	Measures the benefit of deadline-aware queue and prefill decisions.
C. Full three-hook controller	Adds explicit KV demotion and proactive restoration.
D. Oracle ETA experiment	Uses exact recorded tool completion times to establish the upper bound of perfect prediction.

Primary workload

- Critical support agent: alternates between inference and tool waits; target measured from tool completion to first generated token.
- Coding agent: normal priority with a 100,000-token prompt and long chunked prefill.
- Reporting task: background request used to fill otherwise available capacity.
- Run controlled tool durations, early completions, late completions, cancellations, and ETA errors.

Metrics

Metric	Why it matters
Tool-completion-to-first-token P50/P95/P99	Direct measure of resumed-agent responsiveness.
Deadline success rate	Primary SLO result for admitted critical requests.
Background tokens or tasks per GPU-hour	Shows the cost of deadline protection.
HBM KV tokens freed during tool waits	Measures reclaimed fast-memory capacity.
KV restoration wait on critical path	Shows whether prefetch removed visible transfer delay.
Prompt tokens recomputed	Measures failures to retain or restore valuable KV.
Controller overhead	Ensures the policy does not become a new bottleneck.
Command acceptance and expiry	Reveals stale timing, unsupported actions, and failure rates.

Required ablations

- Disable demotion while retaining scheduling control.
- Disable prefetch while retaining host copies.
- Vary background prefill chunk size and reservation window.
- Vary tool ETA error from exact to highly uncertain.
- Compare average-based timing with P99-based timing.

12. Safety, failure handling, and definition of done

Core safety invariants

Invariant	Required behavior
No correctness dependence	A failed controller action may reduce performance but must not change model output correctness.
No stale action	Old tool or prefix actions cannot affect a newer session generation.
No indefinite reservation	Every scheduling restriction and KV protection has an expiry.
No unsafe mutation	Only the scheduler thread applies queue and batch changes.
No memory overcommit	SGLang validates HBM and host capacity before accepting prefetch or retention.
No silent failure	Every action emits an accepted, rejected, deferred, expired, or completed outcome.
Normal fallback	Disabling or losing the controller restores stock SGLang behavior.

Key risks and mitigations

Risk	Mitigation
Tool ETA is inaccurate	Use uncertainty windows, completion-imminent events, P99 estimates, and immediate completion overrides.
Demotion causes more traffic than it saves	Require minimum predicted idle time and measure copy cost before acting.
Background throughput collapses	Cap restriction duration and report throughput cost with every SLO result.
High priority is starved by a long kernel	Use chunked prefill and avoid launching large background work near the prepare window.
Prefix identity is stale or ambiguous	Use session generation, token-hash verification, and fail-open recomputation.
Patch becomes hard to maintain	Keep controller logic separate and make the SGLang adapter thin, versioned, and feature-flagged.

Version 1 definition of done

- The harness can emit the four lifecycle events for a stable session and prefix identity.
- The local scheduler hook can raise an urgent request, reduce background prefill, and safely yield lower-priority work.
- The KV hook can demote, prefetch, and release one identified prefix with observable outcomes.
- Controller loss or disablement returns the worker to stock behavior without restart or data corruption.
- A repeatable benchmark compares stock, scheduler-only, and full-controller configurations.
- Results include deadline success, latency percentiles, background throughput, HBM reclaimed, and KV restoration delay.

13. Codex kickoff instructions

Recommended first Codex task

Begin with repository inspection and a written integration map. Do not implement the full controller until the exact request, scheduler, priority, preemption, and HiCache paths are verified against the pinned SGLang commit.

Copyable Codex prompt: Phase 0 inspection

Inspect the pinned SGLang repository and produce an integration map for a single-worker agent-aware controller. Trace one request from HTTP ingress through internal request creation, waiting-queue ordering, batch formation, priority preemption, radix-cache matching, HiCache host backup/load-back, and request completion.

Identify exact files, classes, methods, locks, thread/process boundaries, and existing metrics for three proposed hooks:

1. Agent-state event ingestion.
2. Deadline-aware scheduler policy.
3. Targeted KV demote/prefetch/release.

Do not edit production code yet. Return a concrete patch plan, API shapes, test locations, and compatibility risks for this exact commit.

After inspection

Ask Codex to implement the milestones separately. Each milestone should include unit tests, integration tests, metrics, feature flags, and a rollback path. Avoid one large patch that combines request schema, scheduling, cache movement, and benchmarks simultaneously.

Suggested pull-request sequence

PR	Scope
PR 1	Versioned event and command models, state machine, no SGLang behavior change.
PR 2	Local enforcer transport, capability discovery, expiry, idempotency, and metrics.
PR 3	Scheduler metadata and policy callback with queue-priority behavior only.
PR 4	Dynamic background prefill budget and safe preemption policy.
PR 5	Targeted KV demote, prefetch, and release commands.
PR 6	Benchmark harness, ablations, dashboards, and results report.

Decision gate before hardware

Only consider hardware acceleration after the best software version is measured. Hardware is justified if it meets the same deadlines at materially higher load, with less reserved capacity, lower control overhead, or more predictable tail latency.

14. Technical references

- [1] NVIDIA Dynamo - Agent Hints
<https://docs.nvidia.com/dynamo/dev/agents/agent-hints>
 - [2] NVIDIA Dynamo - Priority Scheduling
<https://docs.nvidia.com/dynamo/dev/agents/priority-scheduling>
 - [3] NVIDIA Dynamo - SGLang for Agentic Workloads
<https://docs.nvidia.com/dynamo/dev/knowledge-base/modular-components/backends/sg-lang/agents-on-sg-lang>
 - [4] SGLang - HiCache System Design and Optimization
https://docs.sglang.ai/advanced_features/hicache_design.html
 - [5] SGLang - Server Arguments and Scheduling Controls
https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/server_args.py
 - [6] SGLang - Schedule Policy Implementation
https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/schedule_policy.py
 - [7] SGLang - HiRadixCache Implementation
https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/mem_cache/hiradix_cache.py
 - [8] SGLang RFC - First-Class, Versioned KV Hint Envelope
<https://github.com/sgl-project/sglang/issues/36224>
-

Document status

This proposal defines the Version 1 architecture and implementation plan. Exact internal integration points must be confirmed against the pinned SGLang commit before source changes begin.